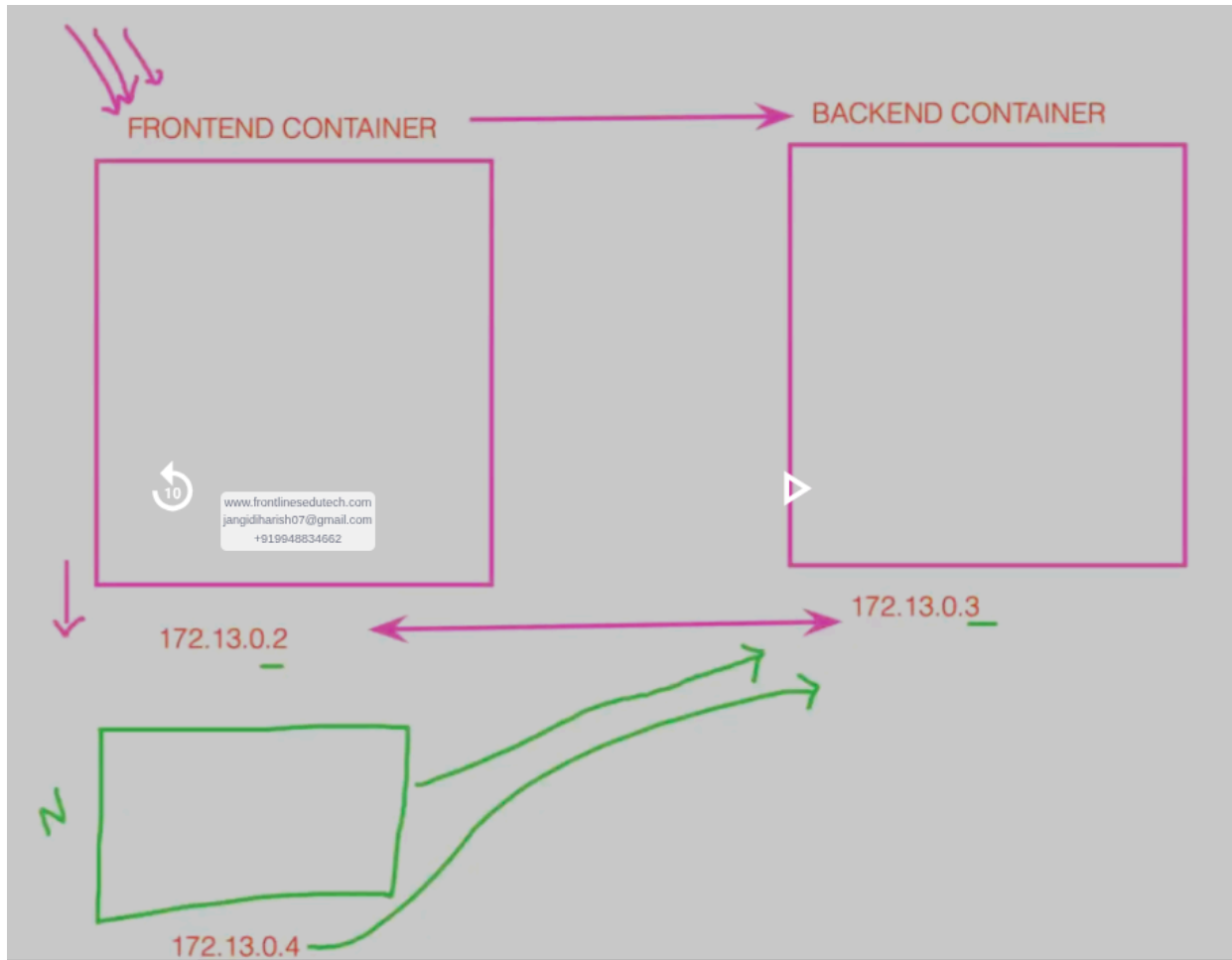


docker_class_8(51)

Docker Network



Generally, one container is connected to another container.

In the 1870s or 1880s, one system was connected to another using wiring systems/connections. Later, it became wireless communication using IP addresses.

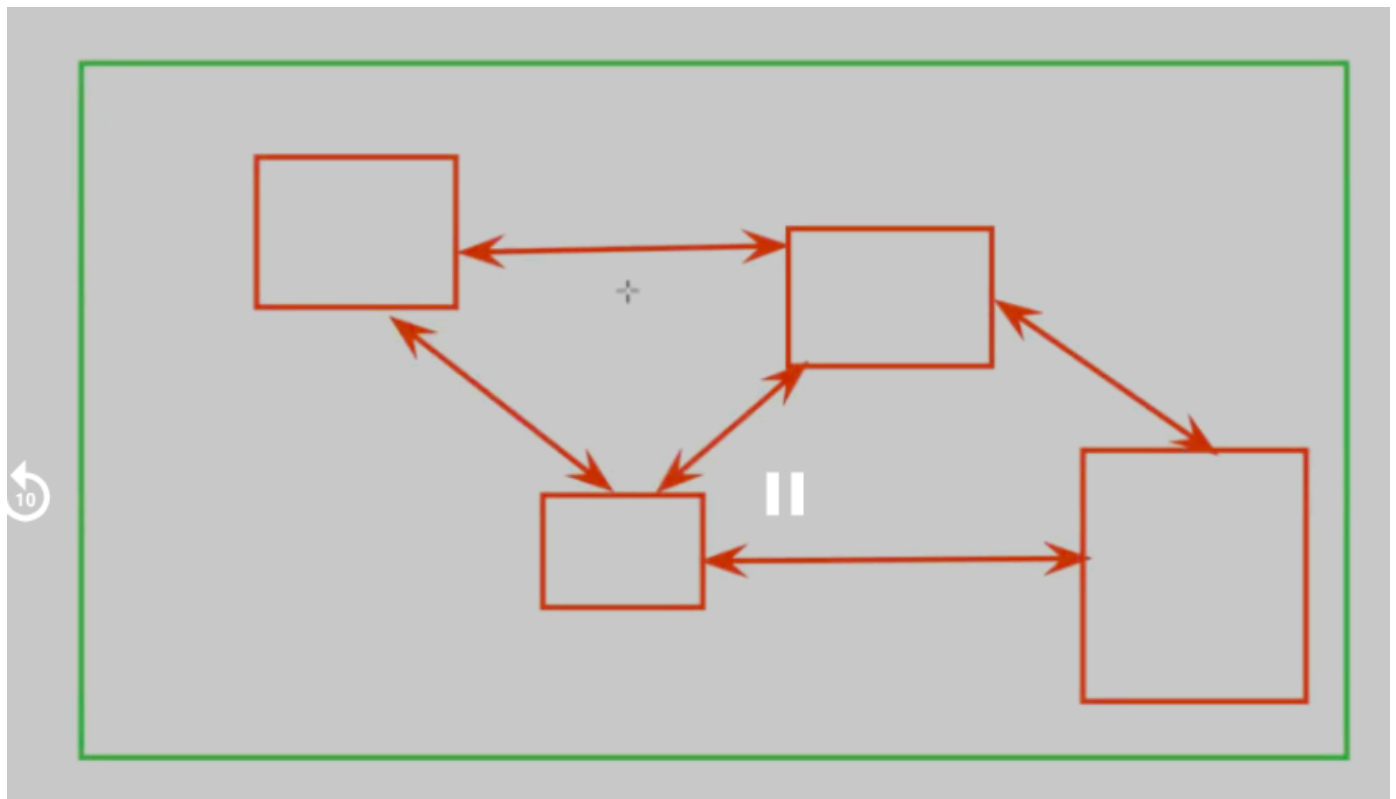
172.13.0.2

172.13.0.3

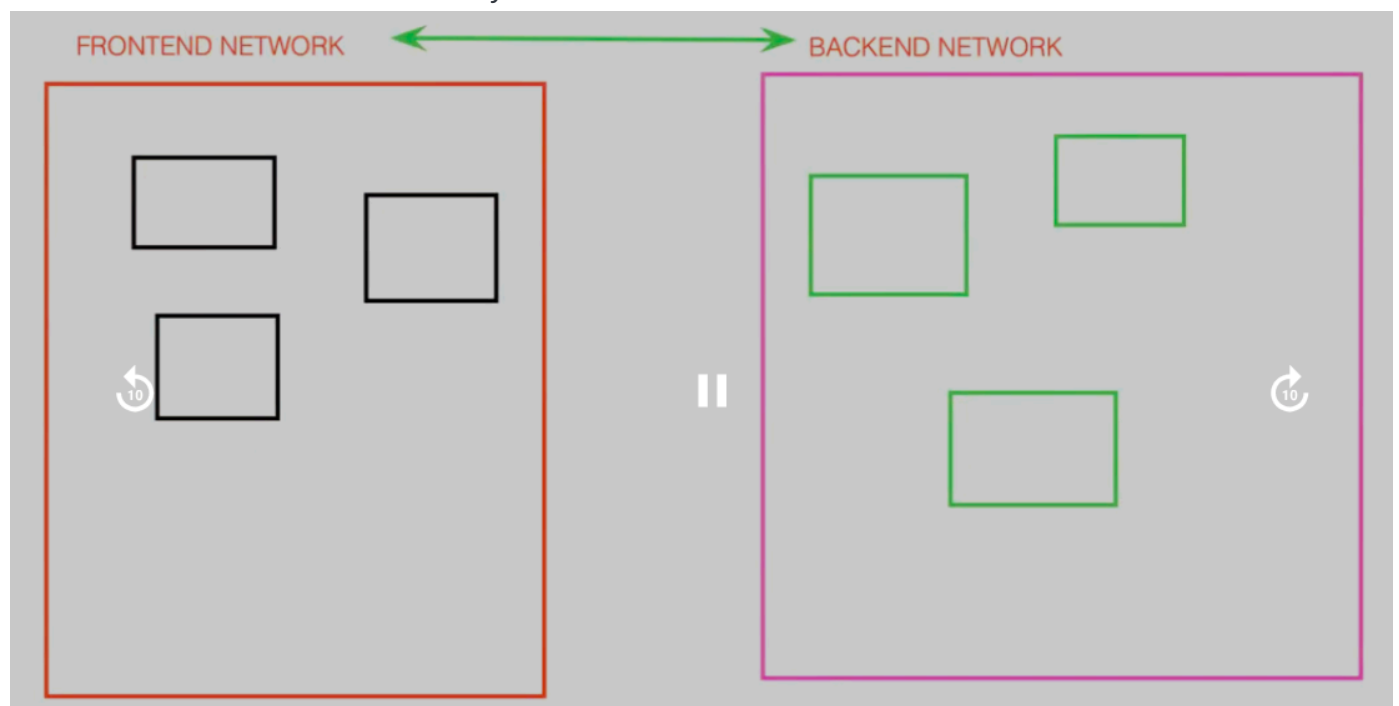
If a container has more load, it will go down, and then the application will also go down. In real time, what we do is: when it goes into downtime, a new container will be created automatically.

Now, the newly created container IP will change to 172.13.0.4. Does it connect to the back-end service? If not, we have to configure it manually. To overcome this, we use a Docker network.

If all containers are using the same network, this issue will not occur, and they will automatically be connected.



the connection it will communicate by network level



If the load increases, then a new container will be created under the network, and it will be easy to communicate.

When the front-end network is connected to the back-end network, all front-end or newly created containers will be created and connected.

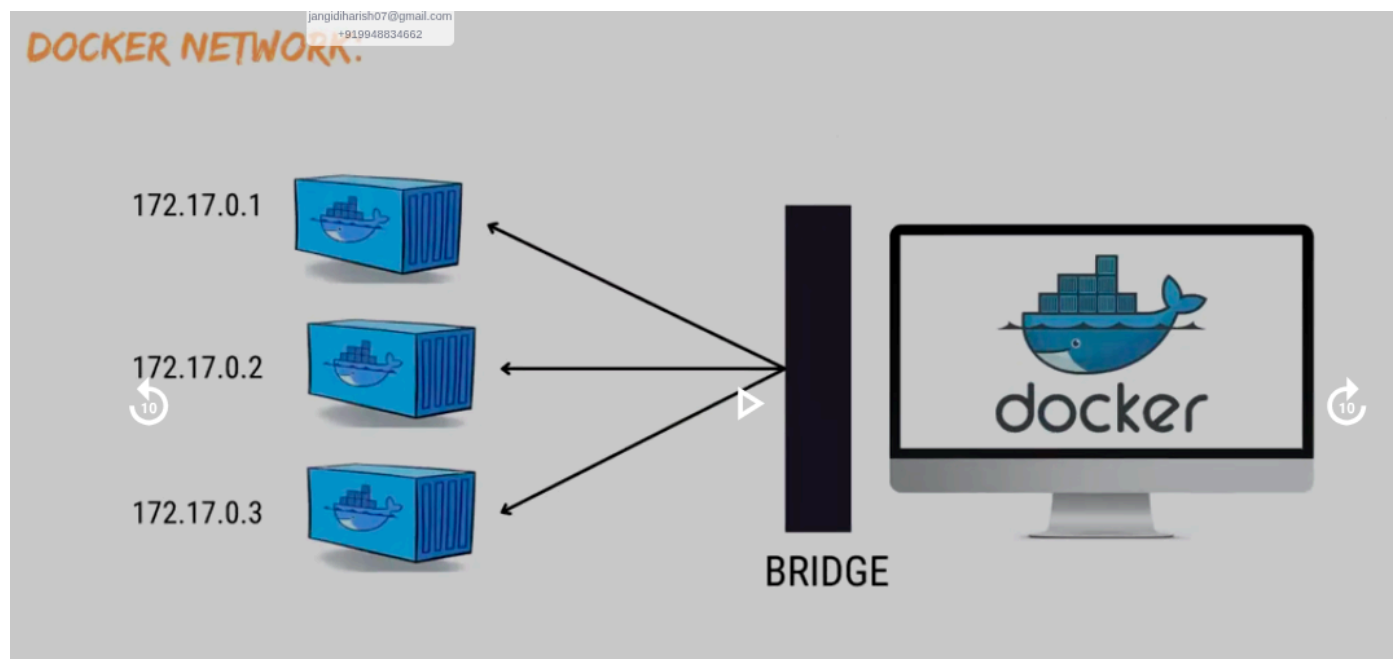
Now, we do not need to do anything manually.

Now this is implemented in real time.

The default network is the bridge network.

While installing Docker, the default network is the bridge network.

The containers will come in sequential order.



This is the order of network sequential ![\[2f1594df355936d10ee7fee3ef0c80cb.png\]](#)
([_resources/23db259765e4432085a22156aefa3e28.png](#))

In one network, up to 65,531 containers can be created.

We can also assign the network IP as per our requirement.

2 power 32 → 65,531

172.14.0.1

172.14.0.2

172.14.0.3

172.14.256.256

This will clearly explain the VPC topic.

WHY DOCKER NETWORK:

Lets assume we are having 2 containers like APP container and DB container. This App container has to communicate with DB container. So the developer will write a code to connect the application to the DB container.

The IP address of a container is not permanent. If a container is removed due to hardware failure, a new container will be created with a new IP, which can cause connection issues.

To resolve this issue we are using docker networks to create our custom network.

Docker networks are used to make a communication between the multiple containers that are running on same or different docker hosts. We have different types of docker networks.

- Bridge Network
- Host Network
- None network
- Overlay network

BRIDGE NETWORK: It is a default network that container will communicate with each other within the same host.

HOST NETWORK: When you Want your container IP and ec2 instance IP same then you use host network

NONE NETWORK: When you don't Want The container to get exposed to the world, we use none network. It will not provide any network to our container.

OVERLAY NETWORK: Used to communicate containers with each other across the multiple docker hosts.

Bridge vs Host: Side-by-Side

BRIDGE NETWORK (what you've been using)

Host Machine

```
|— eth0: 192.168.1.100 (REAL IP)
|— Docker Bridge: 172.10.0.0/16 (VIRTUAL)
    |— postgres: 172.10.0.2
    |— nginx: 172.10.0.3
    |— app: 172.10.0.4
      172.10.255.255
```

Each container has ISOLATED IP

They talk through Docker's virtual network

HOST NETWORK (what you might want for prod)

Host Machine

```
|— eth0: 192.168.1.100 (REAL IP)
|— postgres: DIRECTLY on 192.168.1.100
|— nginx: DIRECTLY on 192.168.1.100
|— app: DIRECTLY on 192.168.1.100
```

NO virtual network—containers share the HOST's network

❌ Bridge Network (Standard)

```
docker run -d \  
  --name postgres-bridge \  
  -p 5432:5432 \  
  postgres:15
```

Access from outside:

```
# psql -h 192.168.1.100 -p 5432
```

Inside other bridge containers:

```
# psql -h postgres-bridge (via Docker DNS)
```

✅ Host Network (Prod-Grade)

```
docker run -d \  
  --network host \  
  --name postgres-host \  
  postgres:15
```

postgres NOW runs on HOST's actual interface

Direct access:

```
# psql -h 192.168.1.100 -p 5432 (NO port mapping needed)
```

Overlay Network = Containers across different hosts communicate as if they are on the same network.

Different server hosts, let's say Server A has container1 and Server B has container2. At this time, we use an overlay network for containers on different servers.

In simple terms, when one server container communicates with another server container, we use an overlay network.

Overlay network will be enabled only when using Docker Swarm.

```
docker run -itd --name cont-1 -p 8080:8080 ubuntu  
docker ps -a
```

I want to check the cont1 network:

```
docker inspect cont1
```

It should be 172.17.0.1 as default.

cont1 should be 172.17.0.2

cont2 should be 172.17.0.3

cont3 should be 172.17.0.4

This is the bridge network.

Q1: Containers on Same Network - Can They Communicate?

YES!  Containers on the SAME network CAN communicate with each other.

Same Network = Containers Talk to Each Other

Create ONE network

```
docker network create app-net
```

Start Container 1

```
docker run -d --name postgres-db --network app-net postgres:15
```

Start Container 2

```
docker run -d --name my-app --network app-net your-app:latest
```

Start Container 3

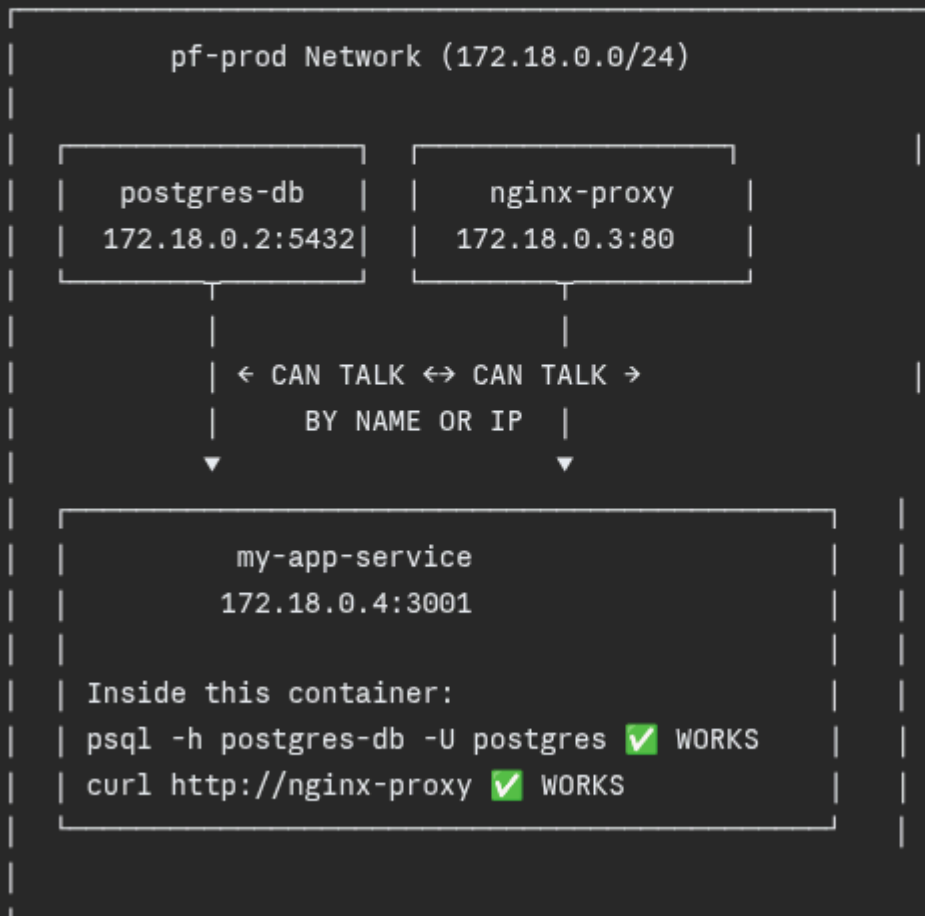
```
docker run -d --name nginx-proxy --network app-net nginx:latest
```

Now:

- `postgres-db` ↔ `my-app`  **Can communicate**
- `my-app` ↔ `nginx-proxy`  **Can communicate**
- `nginx-proxy` ↔ `postgres-db`  **Can communicate**

Visual Example (Your Art of Living Setup)

```
docker network create pf-prod --subnet 172.18.0.0/24
```



Your Current Setup (Art of Living)

```
bash

docker network create pf-prod --subnet 172.18.0.0/24

# PostgreSQL
docker run -d \
  --name postgres-tptm \
  --network pf-prod \
  -v pgdata_tptm_prod:/var/lib/postgresql/data \
  postgres:15

# Nginx
docker run -d \
  --name nginx-reverse-proxy \
  --network pf-prod \
  nginx:latest

# PM2 App Services (all on same network)
docker run -d \
  --name rx-service \
  --network pf-prod \
  your-pm2-image:latest

docker run -d \
  --name bx-fe \
  --network pf-prod \
  your-pm2-image:latest
```

Inside this setup:

- ✓ rx-service CAN reach postgres-tptm
- ✓ bx-fe CAN reach nginx-reverse-proxy
- ✓ nginx CAN reverse proxy to rx-service ↓ bx-fe
- ✓ All services discover each other by NAME

When the front-end network is connected to the back-end network, then all front-end or newly created containers will be created and connected.

In one network, do the containers communicate with each other? Yes.

How to check means using ping.

ping system IP

First, check container 1 (172.17.0.2) and container 2 IP (172.17.0.3).

First, go to cont1:

docker attach cont1

Let's say I am in cont1.

I am in cont1, but I am checking cont2 connection, i.e., ping cont2 IP.

Before that, install ping in cont1, i.e., apt install iputils-ping

Now give:

ping 172.17.0.3

```
root@a20ec4f7d785:/# ping 172.17.0.3
PING 172.17.0.3 (172.17.0.3) 56(84) bytes of data.
64 bytes from 172.17.0.3: icmp_seq=1 ttl=127 time=0.070 ms
64 bytes from 172.17.0.3: icmp_seq=2 ttl=127 time=0.055 ms
64 bytes from 172.17.0.3: icmp_seq=3 ttl=127 time=0.055 ms
```

If packets are going, it means the connection is there.

Now, if we want to check, enter cont2. Before that, install ping and run apt update, then check.

ping 172.17.0.2

```
ping: 172.17.0.2: Name or service not known
root@9f71c60be312:/# ping 172.17.0.2
PING 172.17.0.2 (172.17.0.2) 56(84) bytes of data.
64 bytes from 172.17.0.2: icmp_seq=1 ttl=127 time=0.046 ms
64 bytes from 172.17.0.2: icmp_seq=2 ttl=127 time=0.054 ms
64 bytes from 172.17.0.2: icmp_seq=3 ttl=127 time=0.053 ms
64 bytes from 172.17.0.2: icmp_seq=4 ttl=127 time=0.053 ms
```

ctl+c to stop

so now we will see will create one network and container there we can check

docker network create flm

docker network ls

```
harish@aold2402006:~$ sudo docker network create flm
fdc88b5339f8dff08afc2dfd62656c1d8f2e9dbd73944457df88ec3276fd8c30
harish@aold2402006:~$ docker network ls
NETWORK ID      NAME      DRIVER      SCOPE
042b9cbe8c78    bridge    bridge      local
fdc88b5339f8    flm       bridge      local
8a896ac061d6    flm       bridge      local
f353d40b6451    host      host        local
ce6fb1e28578    none      null        local
harish@aold2402006:~$
```

to know more information about network

docker network inspect ls we can see all details in this we will create containers

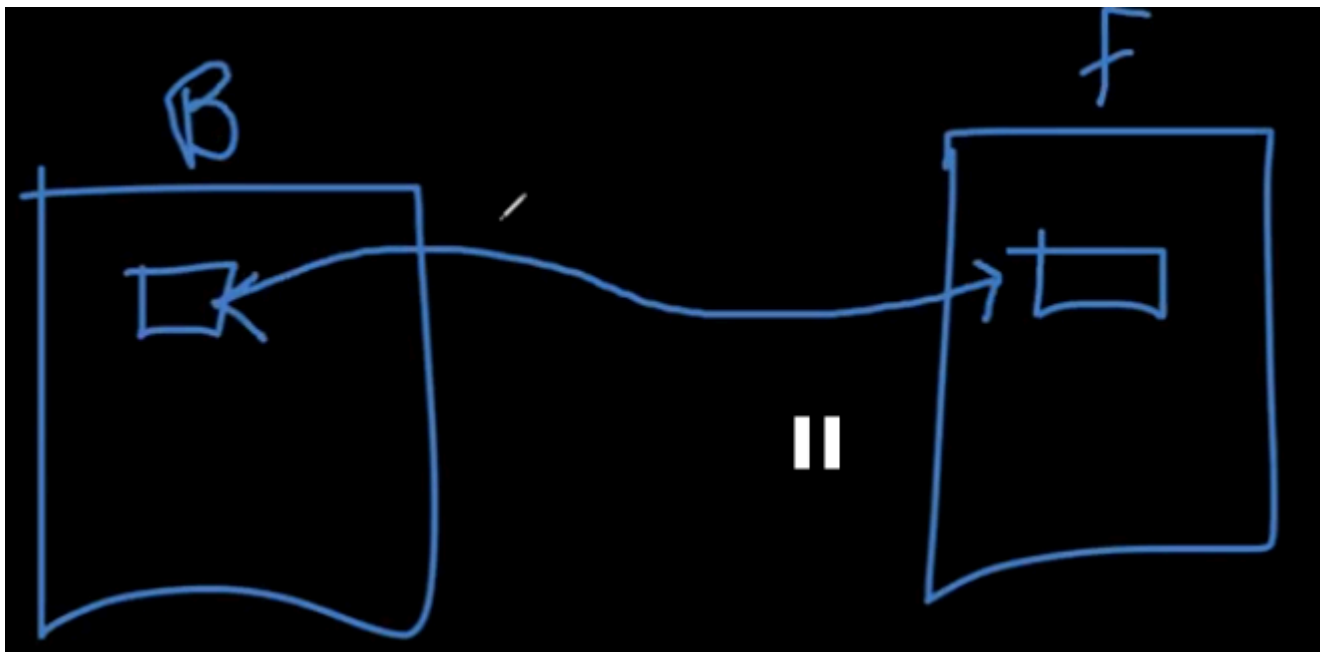
docker run -it --name flm-cont1 --network flm ubuntu

here if we dont give network it will go to bridge

the container will create if we want to know more do

****docker inspect flm-cont1**

Q1: WE HAVE BRIDGE NETWORK AND FLM NETWORK BOTH HAVE CONTIANERS WILL THEY COMMUNUCATE ?? Ans: No.



will go 1st flm-container

docker attach flm-cont1 (this is cont3ip)

install ping and update

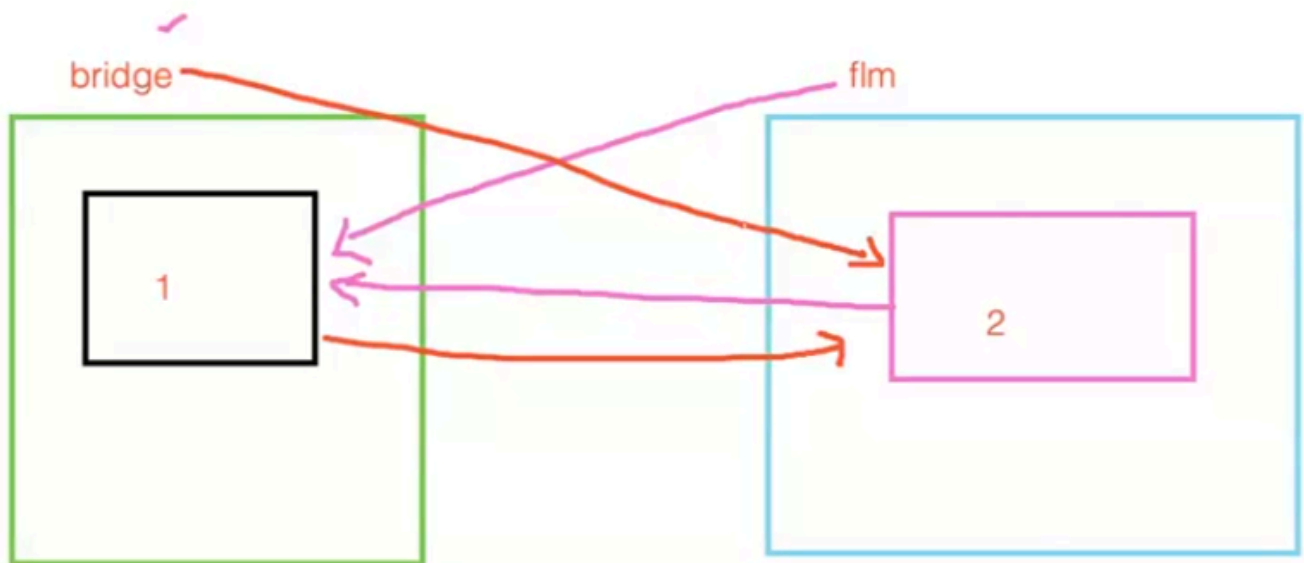
ping 172.17.0.3(cont2ip)

no connection are there it will show struck how to resolve means will see

how to resolve means i will tell

flm-cont1 have flm network

cont1 and cont2 have bridge network and flm-cont1 is flm network



When the flm network is attached to the bridge, then the network will go from the flm network to the bridge, and it will not go from the bridge to the flm network.

If you observe the diagram, cont1 and cont2 are two containers.

If cont2 is the front-end, it has to communicate with cont1. In this case, we have to attach the flm network of cont2 to cont1, then communication will go from cont2 to cont1.

Similarly, if cont1 needs to communicate with cont2, then cont1 network should be attached to cont2.

```
[root@ip-172-31-23-243 ~]# docker network ls
NETWORK ID          NAME       DRIVER  SCOPE
c3751ebe824a        bridge    bridge  local
a0b19713915f        flm        bridge  local
fc7b30826609        host      host    local
c89ecfaf628b        none      null    local
[root@ip-172-31-23-243 ~]# docker network connect flm cont2
```

First, check the network.

Now, the flm network has flm-cont (172.18.0.3). That means I am attaching this network to cont2.

flm-cont is able to communicate with cont2.

The command is:

```
docker network connect flm cont2
```

Earlier, while doing inspect on cont2, we had only one network. Now it will have 2 networks.

```
docker inspect cont2
```

now

```
docker attach flm-cont1
```

```
ping cont2ip
```

now it should work

`cont2` is now on:

bridge

flm

So it has **2 IPs**:

You tested: flm-cont to cont2 But did you test: from cont2 → cont3 👉 It should also work if both are on

`flm`.

cont2 has 2 networks

├─ eth0: bridge network

└─ eth1: flm network (ADDED)

Traffic from cont2 will go DIRECTLY to flm-cont1

(not through any middleman)

BEFORE: docker network connect flm cont2

cont2: 1 network (bridge only)

flm-cont1: 1 network (flm only)

Result: ❌ CANNOT reach each other

AFTER: docker network connect flm cont2

cont2: 2 networks (bridge + flm) ✅

flm-cont1: 1 network (flm only)

Result: ✅ CAN reach each other DIRECTLY

When cont2 sends packet to flm-cont1:

cont2 → eth1 (flm network) → flm-cont1

(DIRECT, same network, no middleman)

So BOTH can reach each other:

✅ cont2 → flm-cont1 (via eth1 interface)

✅ flm-cont1 → cont2 (via eth0 interface)

BIDIRECTIONAL COMMUNICATION!

Visual

```
flm Network (172.18.0.0/24)
|
├─ flm-cont1 (eth0: 172.18.0.2)
|   └─ Can ping cont2? ✅ YES
|
└─ cont2 (eth1: 172.18.0.3)
    └─ Can ping flm-cont1? ✅ YES
```

SAME NETWORK = BOTH CAN REACH EACH OTHER

KEY POINT: Both containers are on the **SAME flm network**, so communication is **BIDIRECTIONAL** ✅

docker attach flm-cont1

ping cont2 IP

It will not work because cont2 has flm network and bridge network after attach, but flm-cont1 has only one network, i.e., flm network. For communication from flm-cont1 to cont2, we have to attach:

```
docker network connect bridge flm-cont1
```

```
docker attach flm-cont1
```

```
ping 172.17.0.3
```

It will connect.

Summary

Test	Works?	Why
<code>flm-cont1 ping cont2</code>	✓ YES	Both on flm network
<code>flm-cont1 ping 172.17.0.2</code>	✗ NO	flm-cont1 has no bridge interface
After <code>docker network connect bridge flm-cont1</code>	✓ YES	Now flm-cont1 has bridge interface

If flm-cont1 needs to reach cont2's **bridge IP address**, then it needs the bridge network attached.

But: They can already reach each other via the **flm network** (which they share). No need for anything else unless you specifically need the bridge IP. ✓

```
Test Results

bash

# Test 1: Reach via flm network
docker exec flm-cont1 ping cont2
✓ WORKS (using flm network)

# Test 2: Reach cont2's bridge IP
docker exec flm-cont1 ping 172.17.0.2
✗ FAILS (flm-cont1 has no bridge interface)

# Test 3: IF you want this to work, add bridge to flm-cont1
docker network connect bridge flm-cont1
docker exec flm-cont1 ping 172.17.0.2
✓ NOW WORKS (flm-cont1 has bridge interface)
```

The container stays on the **NETWORK**, not the IP!

so this way docker networks will work

To know the commands of Docker network:

To create a network, we use:

`docker network create`

If we want to connect a container to a network, then we use:

`docker network connect`

If we want to disconnect, we use:

`docker network disconnect network_name container_name`

Example: cont2 has 2 networks. To remove cont2 from flm network, we use:

`docker network disconnect flm cont2`

To remove unused networks, we use:

`docker network prune`

To remove a network, we use:

`docker network rm network_name`

Basic earlier one task has pending i.e

through the jenkins pipeline i have to create image and container and tag and push to docker registry and then later run the application .

for that we have to install **docker pipeline** plugin

```
pipeline {
    agent any

    stages {
        stage('Code') {
            steps {
                git 'https://github.com/devops0014/tic-tac-toe-docker.git'
            }
        }

        stage('Build') {
            steps {
                sh 'docker build -t image1 .'
            }
        }
    }
}
```

RUN and check and give full permissions i.e `chmod 777 /var/run/docker.sock` and build check in terminal

and next go to pipeline syntax and select in sample steps select last 2 second option i.e
withDockerRegistry -->select username and password give username and password and id docker hub
select credntials and generate pipline copy and paste

// This step should not normally be used in your script. Consult the inline help for details.

```
withDockerRegistry(credentialsId: 'dockerhub') {  
    // some block  
}  
  
pipeline {  
    agent any  
  
    stages {  
        stage('Code') {  
            steps {  
                git 'https://github.com/devops0014/tic-tac-toe-docker.git'  
            }  
        }  
  
        stage('Build') {  
            steps {  
                sh 'docker build -t image1 .'  
            }  
        }  
  
        stage('Docker Registry') {  
            steps {  
                script {  
                    docker.withRegistry("", 'dockerhub') {  
                        sh 'docker tag image1 shaikmustafa/leo:vijayimage'  
                        sh 'docker push shaikmustafa/leo:vijayimage'  
                    }  
                }  
            }  
        }  
    }  
}
```

build now and check in docker registry it is there
for tin this i will create a container

```
pipeline {  
    agent any
```

```

stages {
  stage('Code') {
    steps {
      git 'https://github.com/devops0014/tic-tac-toe-docker.git'
    }
  }

  stage('Build') {
    steps {
      sh 'docker build -t image1 .'
    }
  }

  stage('Docker Registry') {
    steps {
      script {
        docker.withRegistry("", 'dockerhub') {
          sh 'docker tag image1 shaikmustafa/leo:vijayimage'
          sh 'docker push shaikmustafa/leo:vijayimage'
        }
      }
    }
  }

  stage('Deploy') {
    steps {
      script {
        // stop & remove old container if exists
        sh ""
        docker rm -f myapp || true
        docker run -d --name myapp -p 9900:80 shaikmustafa/leo:vijayimage
        ""
      }
    }
  }
}

```

save and build happily container will will deploy